

Retrieval Augmented Generation



Empowering with Disruptive Technologies



Drawbacks of directly using LLMs

- Adaptability and Maintenance
- Scalability Across Multiple Domains
- Hallucinations
- Knowledge cutoffs

What is a possible solution?

Human behavioral analogy:

Given a query,

We do not have all the knowledge in the world. Often human experts know where to find the relevant information and then answer based on the retrieved relevant information.

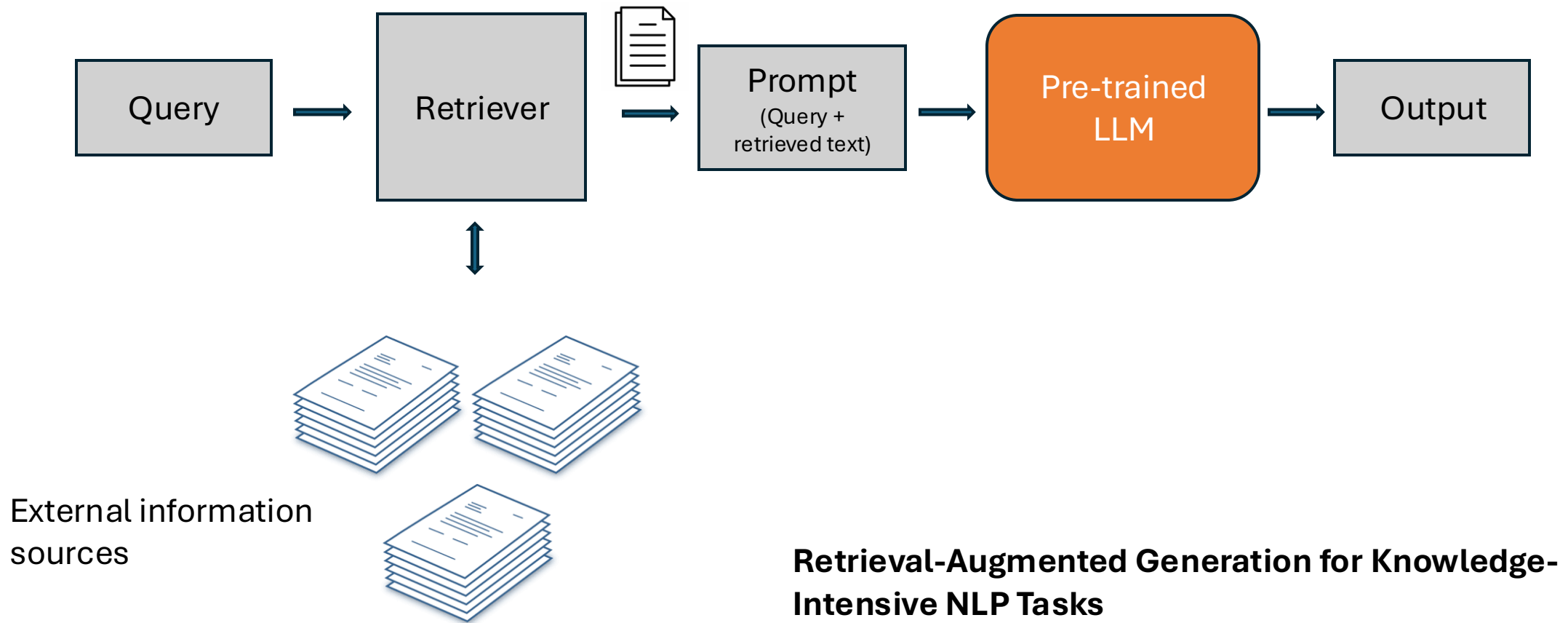
Humans have two models

1. Language model
2. Information retrieval model

Let's try a similar architecture in Gen-AI



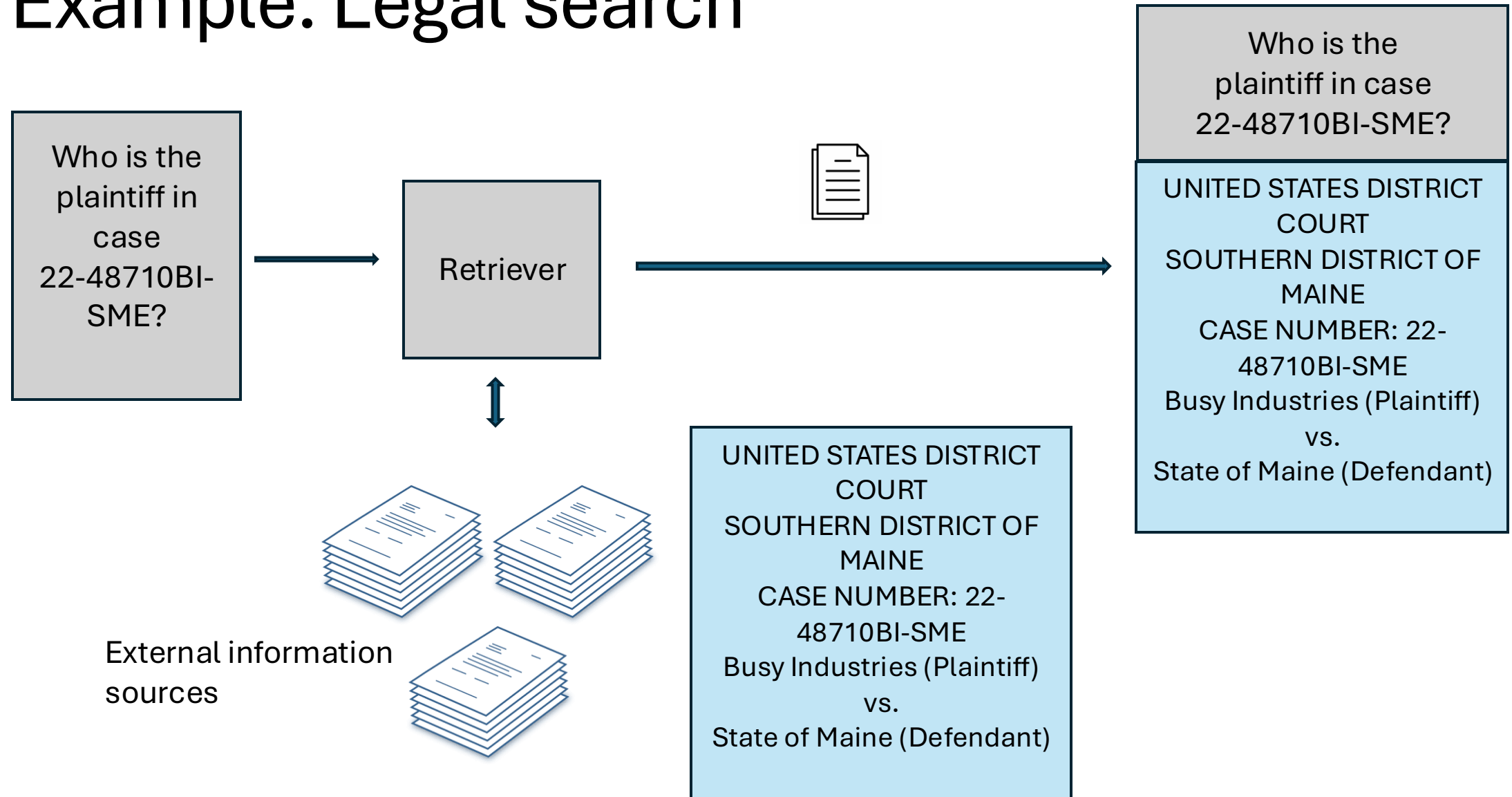
Retrieval augmented generation (RAG)



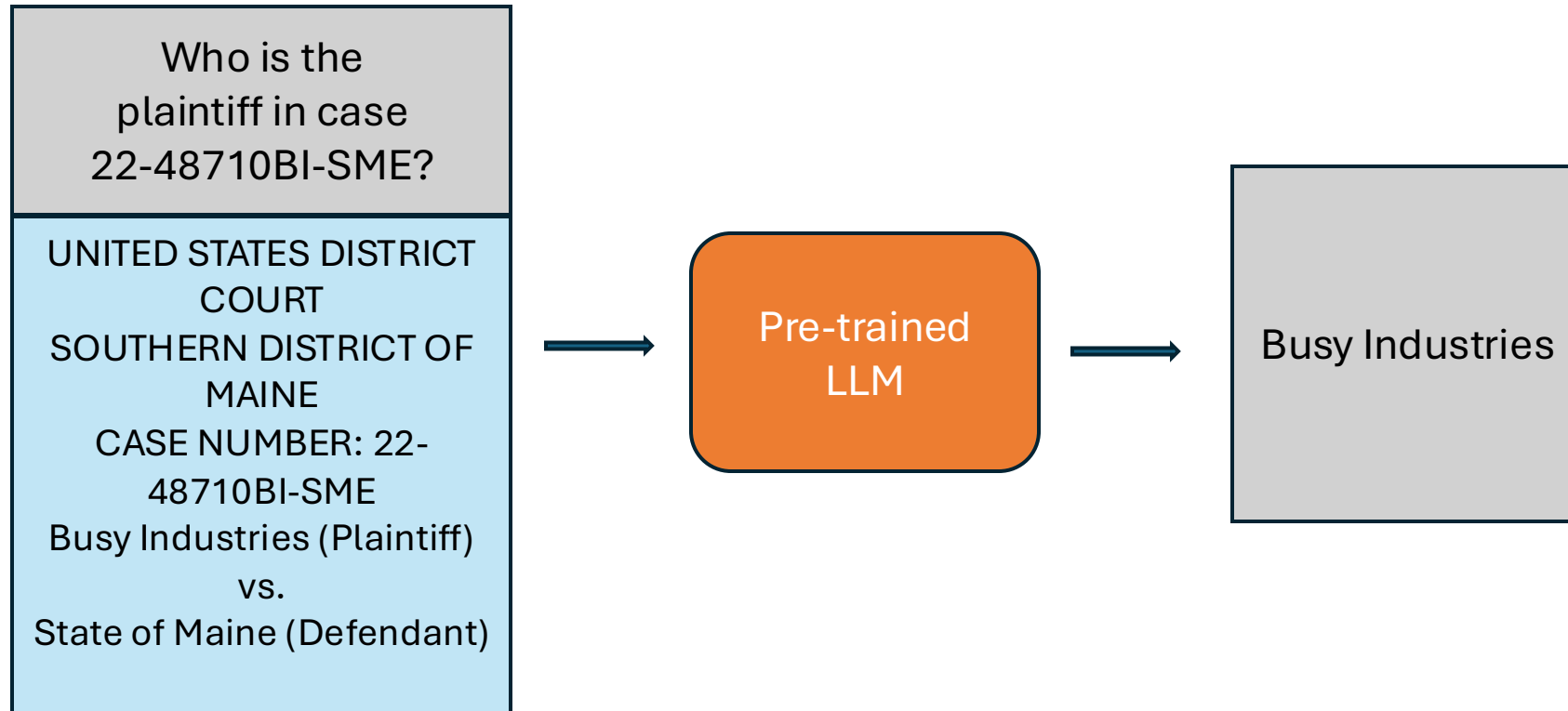
Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

<https://arxiv.org/abs/2005.11401>

Example: Legal search



Example: Legal search



RAG can integrate with many sources

External information sources

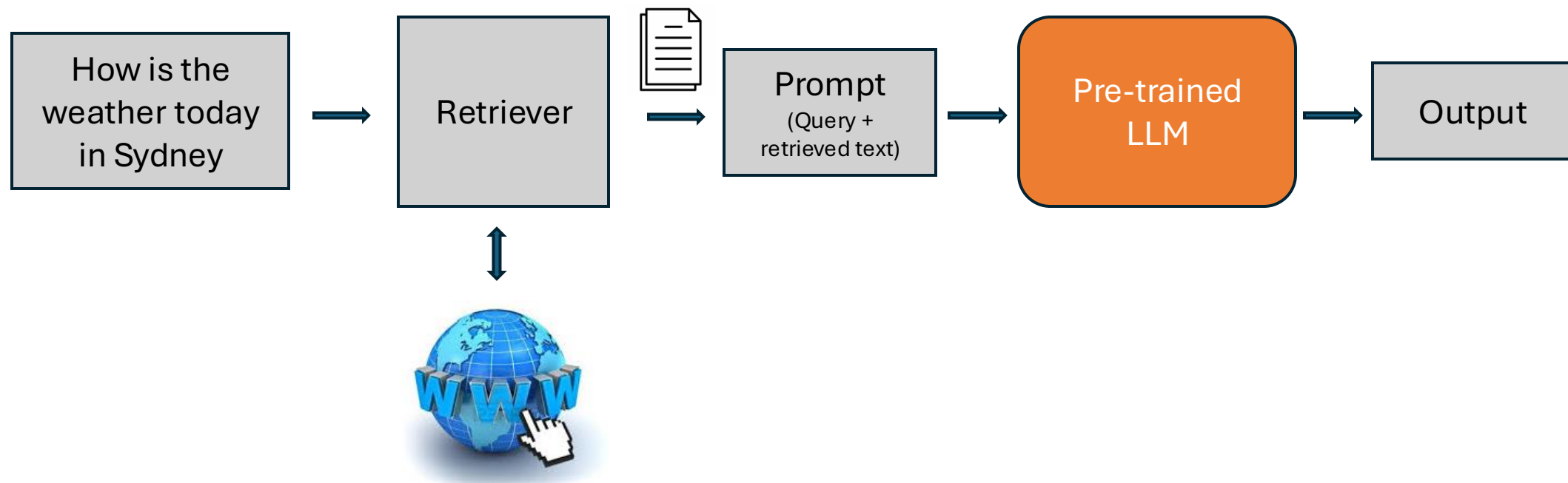
- Documents
- Wikis
- Expert Systems
- Web pages
- Databases
- Vector Store

Need to create an appropriate **Retriever**

RAG can integrate with many sources

For information on day-to-day news like weather, latest news summary, etc.

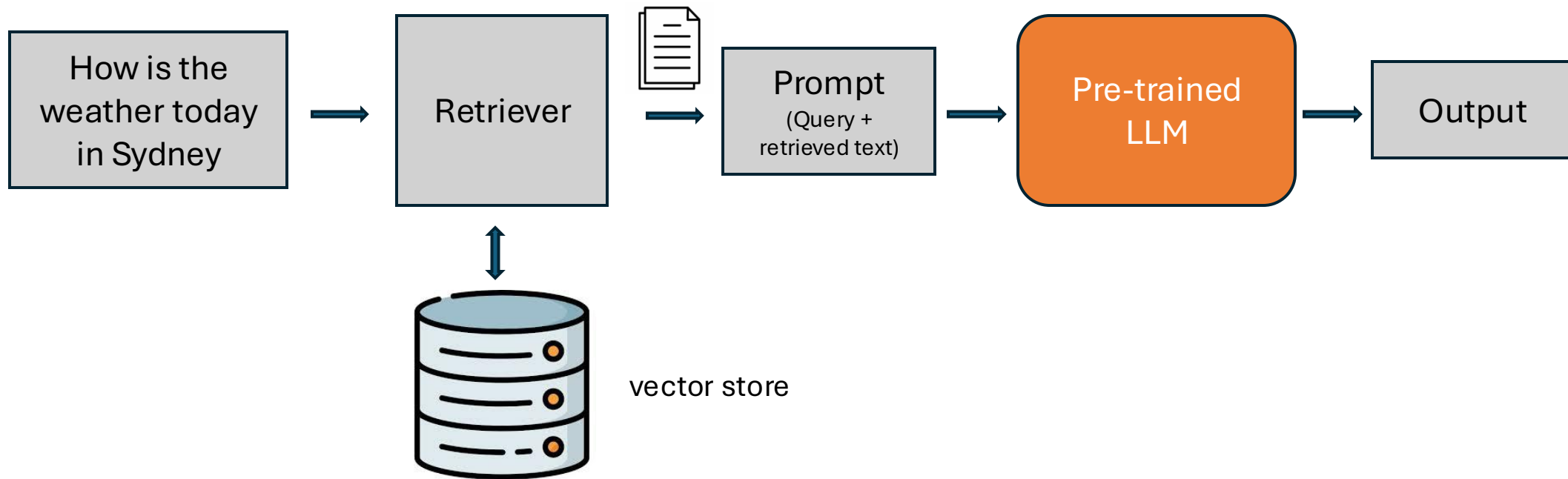
RAG can be integrated with world-wide-web via a search engine.



RAG can integrate with many sources

For information from specific set of knowledge articles (or text repository)

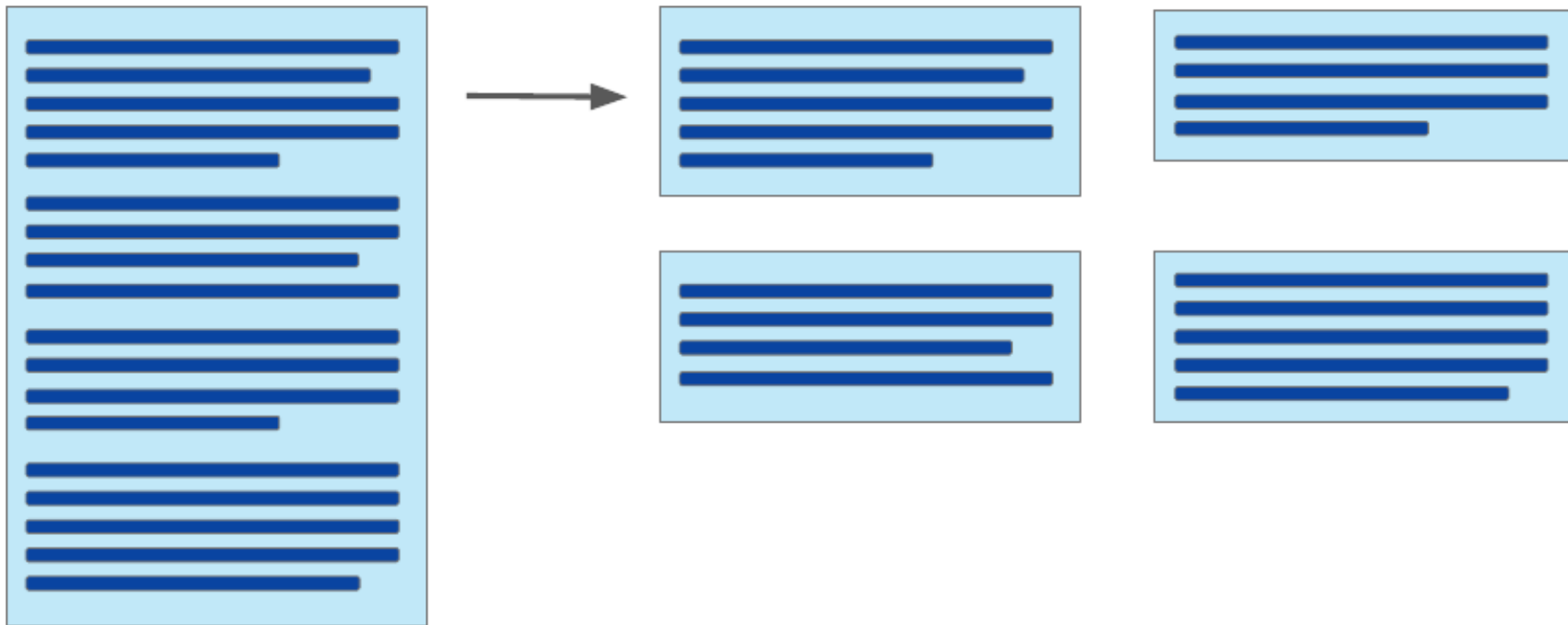
RAG can be integrated with vector store.



How is a vector store created?

STEP-1

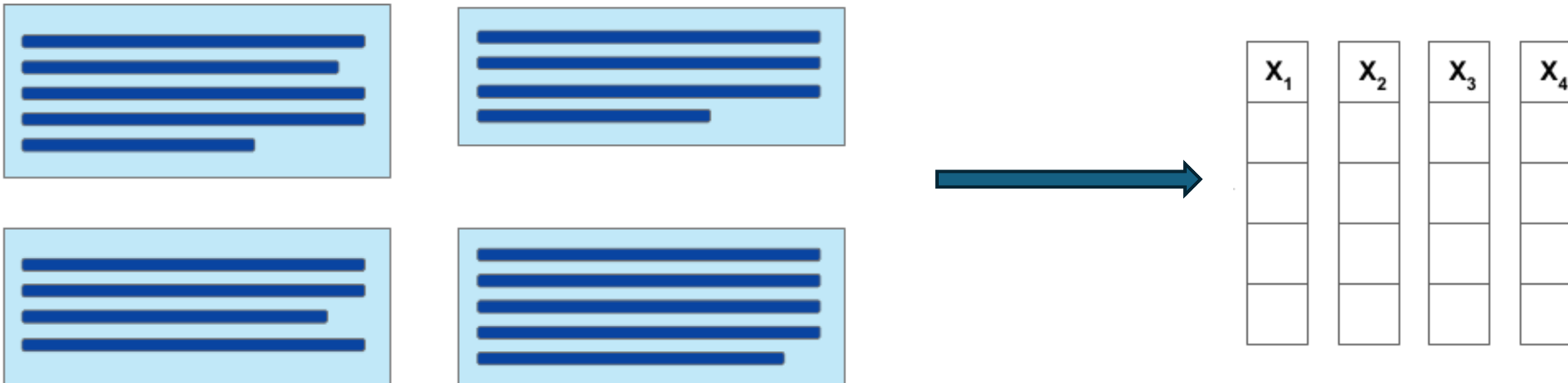
- Split the document into multiple chunks.
 - Helps retrieve sections specific to the query instead of entire document.
 - Also, help with context length limitation of the LLM



How is a vector store created?

STEP-2

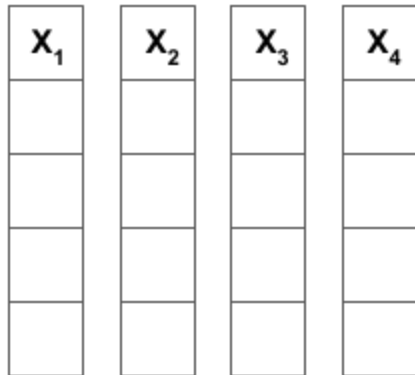
- Encode each chunk into a vector embedding
 - Use text-encoder models like Sentence Transformer or a dense retriever like DPR (Dense Passage Retriever)
 - Vector represents semantic meaning of the text.



How is a vector store created?

STEP-3

- Store the embedding vectors in a vector store like FAISS.
 - Helps in quick similarity computation with query vector.
 - Optimized storage and indexing



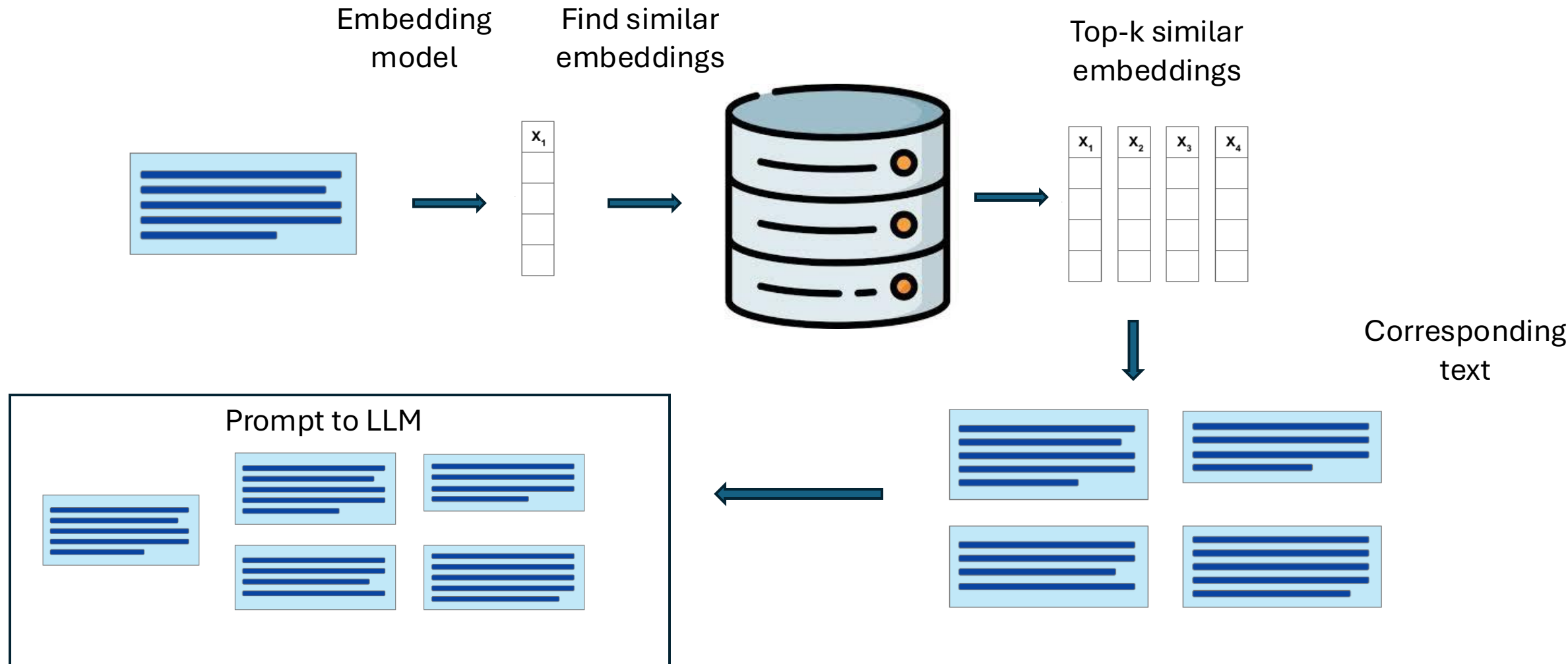
vector store

Constraints during embeddings creation.

Two considerations for using external data in RAG:

- Data must fit inside context window, decide chunk size accordingly.
- Data must be in format that allows its relevance to be assessed at inference time: Embedding vectors

Inference



Challenges of using RAG

- Dependency on Retrieval Quality
 - The response is as good as the retrieved text.
 - Good embedding model is essential
- Response Length Constraints
 - Return of large text requires further processing to summarize to fit into context length of LLM.
- Complexity and Resource Needs
 - Need to maintain a vector database
 - Document management system.
 - additional computational and infrastructure requirements

Benefits of using RAG

- Dynamic Knowledge Updating
 - No necessity to finetune model for new data
 - Data can be added to the RAG dynamically
- Reduced Hallucination
 - Because it grounds the generation in specific, retrieved documents.
- Scalability Across Domains
 - Can be used across multiple domains
 - Different databases can be used for different tasks
 - Single LLM is sufficient.

Interesting read

- **Toward Optimal Search and Retrieval for RAG**

- <https://arxiv.org/pdf/2411.07396>

Toward Optimal Search and Retrieval for RAG

Alexandria Leto

Department of Computer Science
University of Colorado Boulder
alex.letto@colorado.edu

Cecilia Aguerrebere

Intel Labs
cecilia.aguerrebere@intel.com

Ishwar Bhati

Intel Labs
ishwar.s.bhati@intel.com

Ted Willke

Intel Labs
ted.willke@intel.com

Mariano Tepper

Intel Labs
mariano.tepper@intel.com

Vy Ai Vo

Intel Labs
vy.vo@intel.com

Abstract

Retrieval-augmented generation (RAG) is a promising method for addressing some of the memory-related challenges associated with Large Language Models (LLMs). Two separate systems form the RAG pipeline, the retriever and the reader, and the impact of each on downstream task performance is not well-understood. Here, we work towards the goal of understanding how retrievers can be optimized for RAG pipelines for common tasks such as Question Answering (QA). We conduct experiments focused on the relationship between retrieval and RAG performance on QA and attributed QA and unveil a number of insights useful to practitioners developing high-performance RAG pipelines. For example, lowering search accuracy has minor implications for RAG performance while potentially increasing retrieval speed and memory efficiency.

Vector databases

Goals of Vector Database

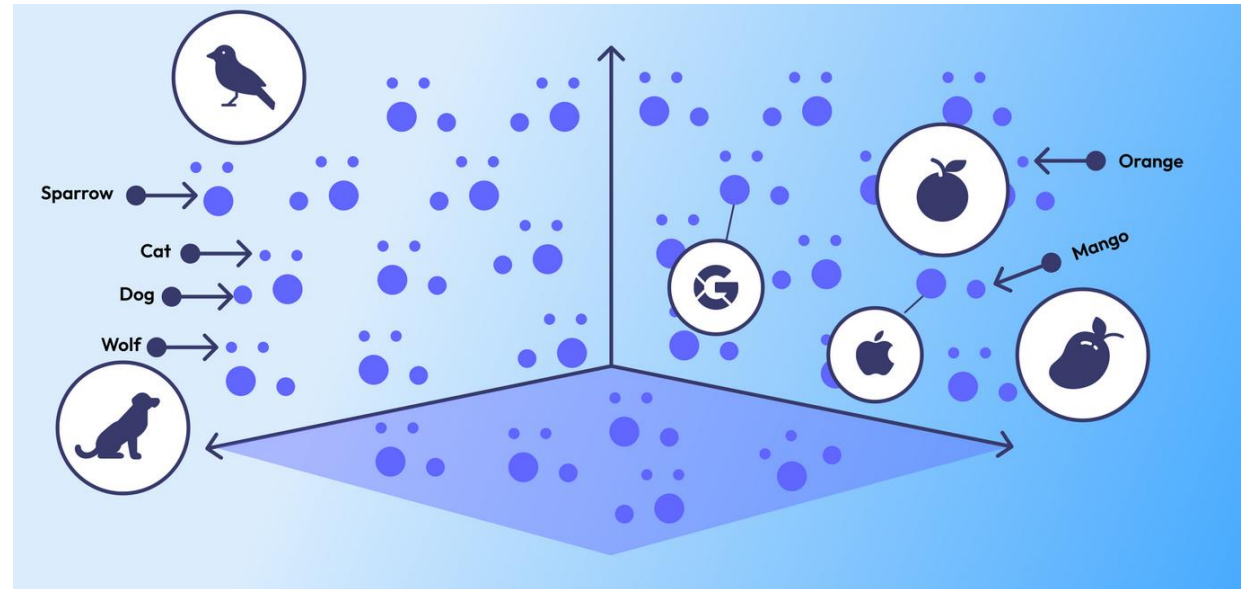
Efficiently,

- **Store**
- **Index**
- **Retrieve** (e.g., similarity computation)

Vector representations of unstructured or structured data.

Scalability for High-Dimensional Data

- Manage large-scale datasets with potentially billions of vectors
- Support distributed storage and processing to accommodate the needs of enterprise-scale applications.



Use-cases for vector database



Retrieval Augmented Generation (RAG)

Expand LLMs' knowledge by incorporating external data sources into LLMs and your AI applications.



Recommender System

Match user behavior or content features with other similar ones to make effective recommendations.



Text/ Semantic Search

Search for semantically similar texts across vast amounts of natural language documents.



Image Similarity Search

Identify and search for visually similar images or objects from a vast collection of image libraries.



Video Similarity Search

Search for similar videos, scenes, or objects from extensive collections of video libraries.



Audio Similarity Search

Find similar audios in large datasets for tasks like genre classification or speech recognition



Molecular Similarity Search

Search for similar substructures, superstructures, and other structures for a specific molecule.



Anomaly Detection

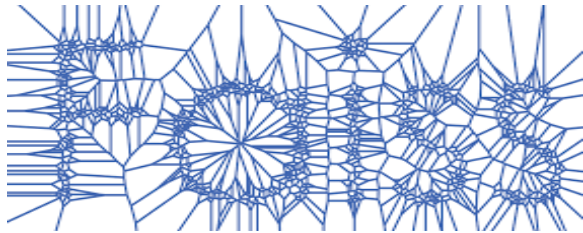
Detect data points, events, and observations that deviate significantly from the usual pattern



Multimodal Similarity Search

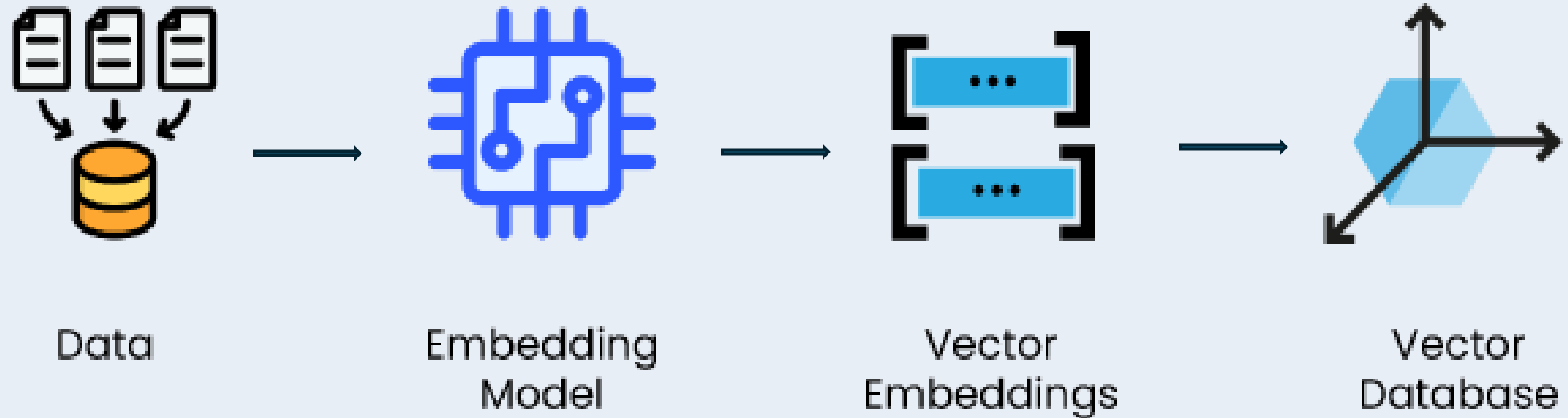
Search over multiple types of data simultaneously, e.g. text and images

Vector databases



Chroma

Where do vectors come from?



Embedding model options

Retrieval is the task of finding relevant documents for a query.

English Chinese French Polish Russian Law LongEmbed RAR-b BRIGHT BRIGHT Long CoIR

Retrieval English leaderboard

◦ Metric: Normalized Discounted Cumulative Gain @ 10 (nDCG@10)

◦ Languages: English

Rank	Model	Model Size (Million Parameters)	Memory Usage (GB, fp32)	Embedding Dimensions	Max Tokens	Average	ArguAna	ClimateFEVER	CQADupstackRetrieval	DBPedia
1	NV-Embed-v2	7851	29.25	4096	32768	62.65	70.07	45.39	50.24	53.5
2	bge-en-icl	7111	26.49	4096	32768	62.16	83.08	45.43	47.31	51.63
3	stella_en_1.5B_v5	1543	5.75	8192	131072	61.01	65.27	46.11	47.75	52.28
4	NV-Retriever-v1	7111	26.49	4096	512	60.9	68.28	43.47	49.36	50.82
5	gte-Qwen2-7B-instruct	7613	28.36	3584	131072	60.25	64.27	45.88	46.43	52.42
6	gte-Qwen2-7B-instruct-GGUF					60.25	64.27	45.88	46.43	52.42
7	Linq-Embed-Mistral	7111	26.49	4096	32768	60.19	69.65	39.11	47.27	51.32
8	SFR-Embedding-2_R	7111	26.49	4096	32768	60.18	62.34	34.43	46.11	51.21
9	Zeta-Alpha-E5-Mistral	7111	26.49	4096	32768	59.5	65.62	42.05	48.85	51.2
10	NV-Embed-v1	7851	29.25	4096	32768	59.36	68.2	34.72	50.51	48.29
11	bge-multilingual-gemma2	9242	34.43	3584	8192	59.24	77.37	39.37	47.94	51.37

◦ Total Datasets: 213

◦ Total Languages: 113

◦ Total Scores: 91696

◦ Total Models: 479

Made with ❤️ for NLP. If this work is useful to you, please consider citing:

<https://huggingface.co/spaces/mteb/leaderboard>

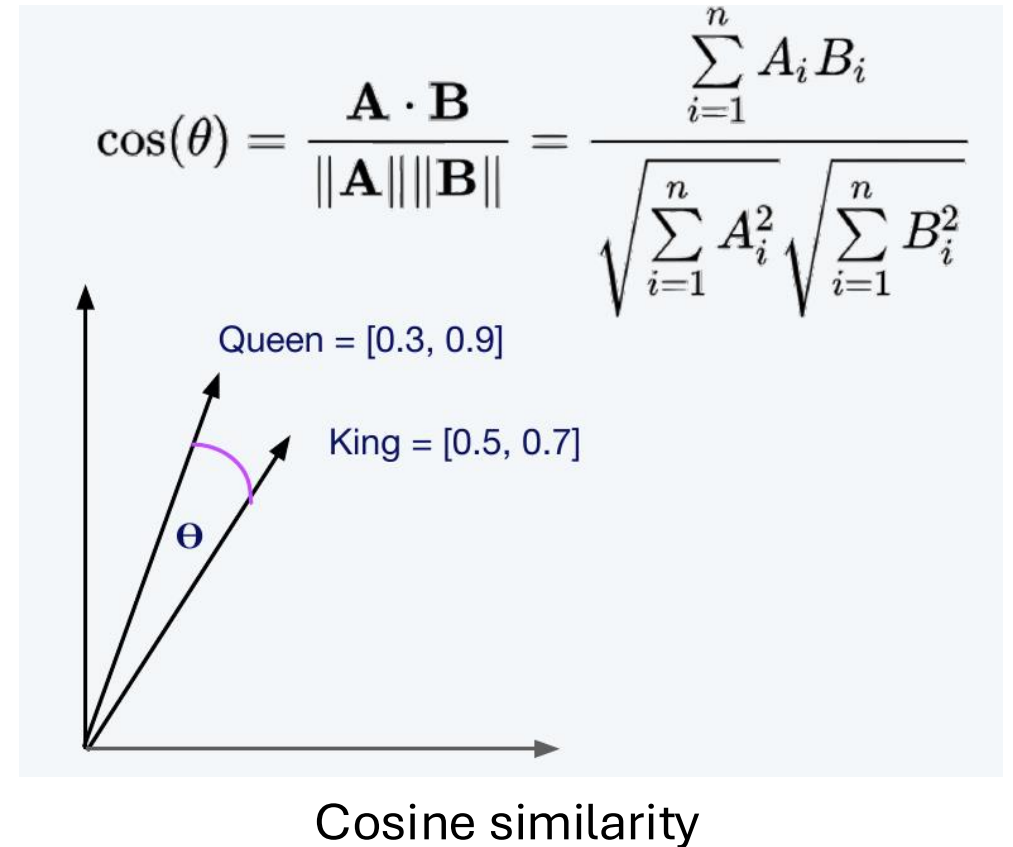
How does it work?

- Given a query (text, image, etc.)
- Query is converted into a vector representation using same embedding model.
- Find the nearest set of vectors in the database for the given query vector.
- This is provided semantically similar text samples, visually similar images samples, etc.

What do we mean by nearest?

Distance metrics for finding nearest vectors or similar vectors.

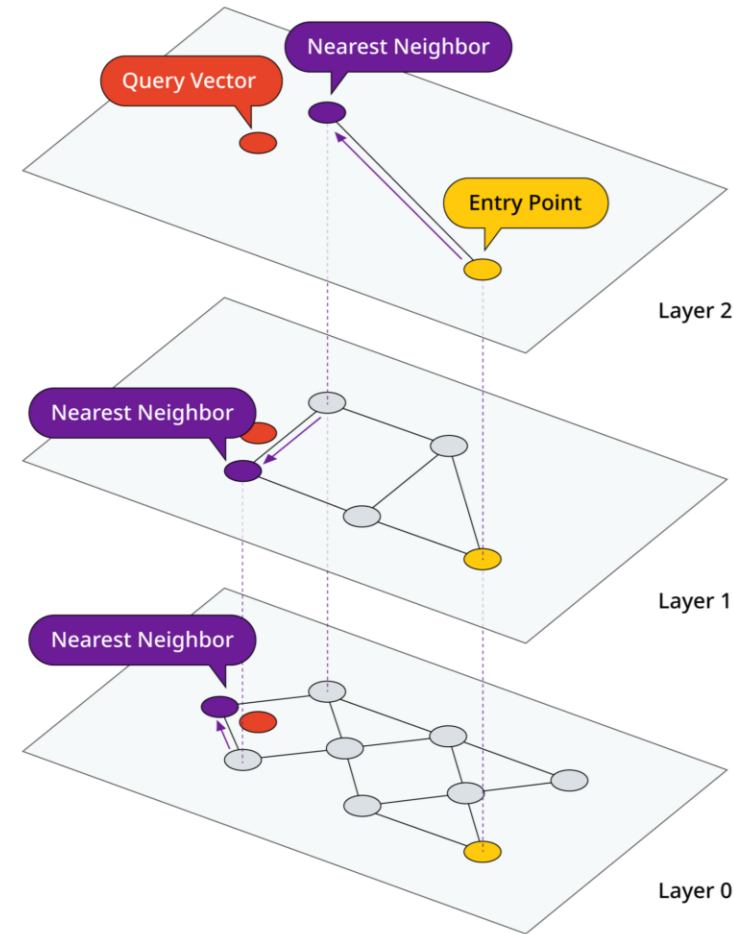
- Euclidian distance
- Inner product
- Cosine similarity



Indexing: For efficient computations

- Hierarchical Navigable Small World (HNSW)
- Product Quantization (PQ)
- Approximate Nearest Neighbor (ANN) Indexing
- Inverted File Index (IVF)
- FAISS Indexing (Facebook AI Similarity Search)

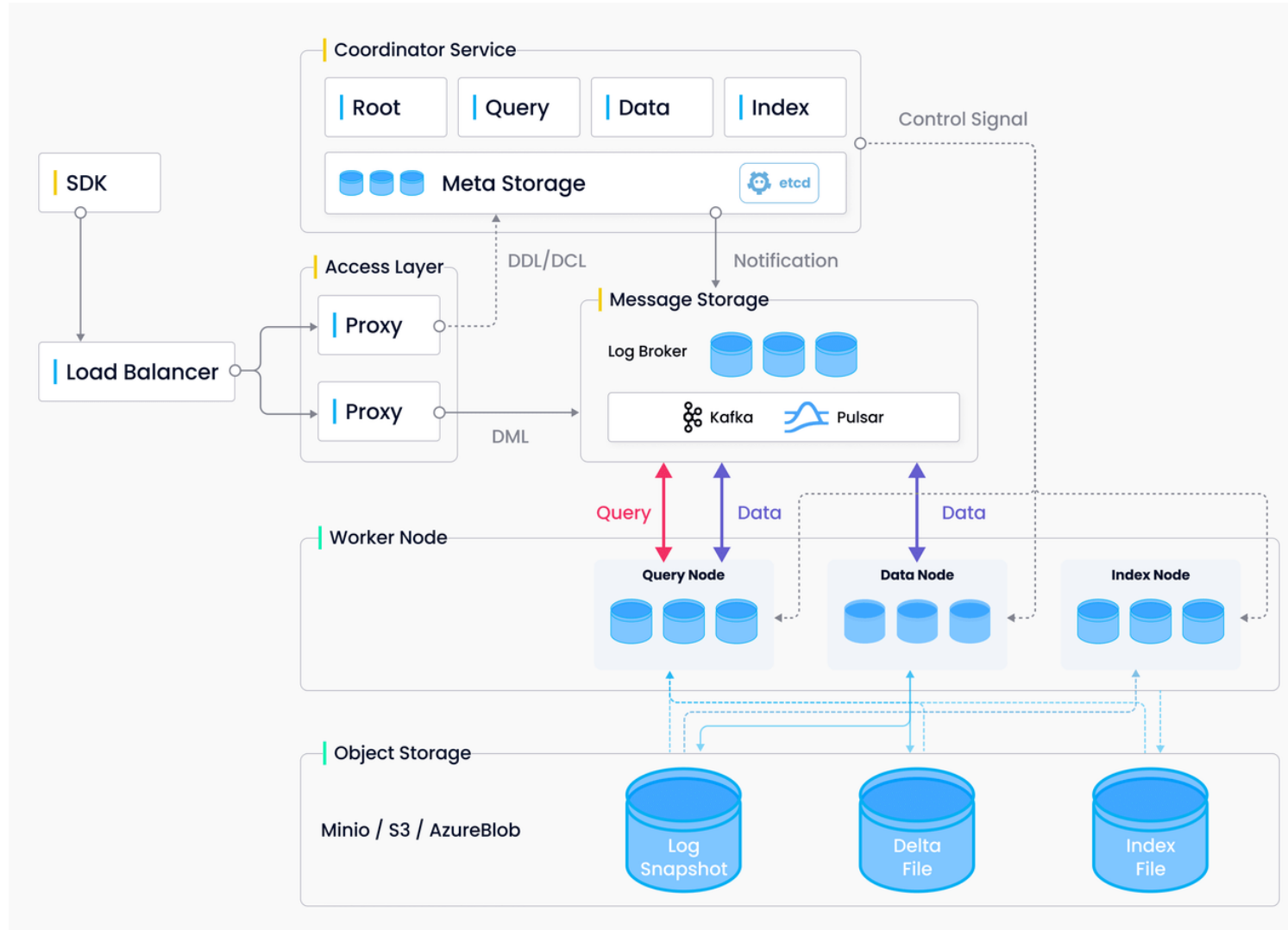
Each can be best of specific scenario.



HNSW

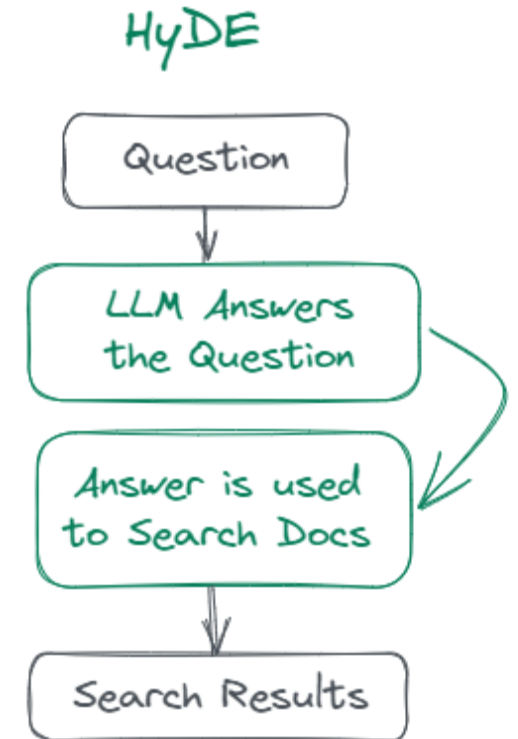
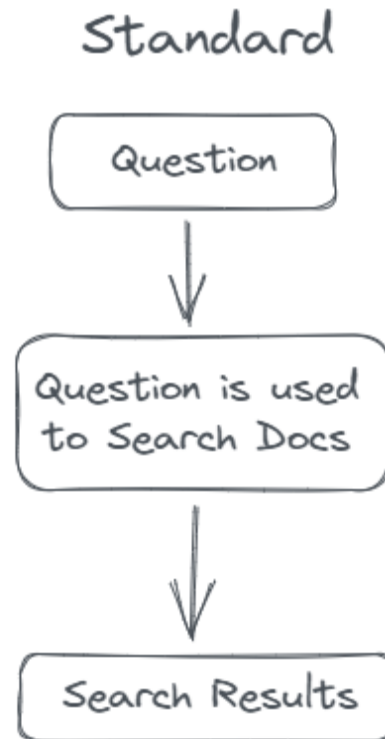
Vector DB architecture

Milvus Architecture



Enhancements in RAG: Query rewriting

- Rewrite the user query such that our RAG will know best how to answer.
- Use a Generative AI model to rewrite the query.
 - Same LLM as that of used in final step.
 - SLM tuned for the task of rewriting.
- Alternative mechanism to rewrite is HyDE.



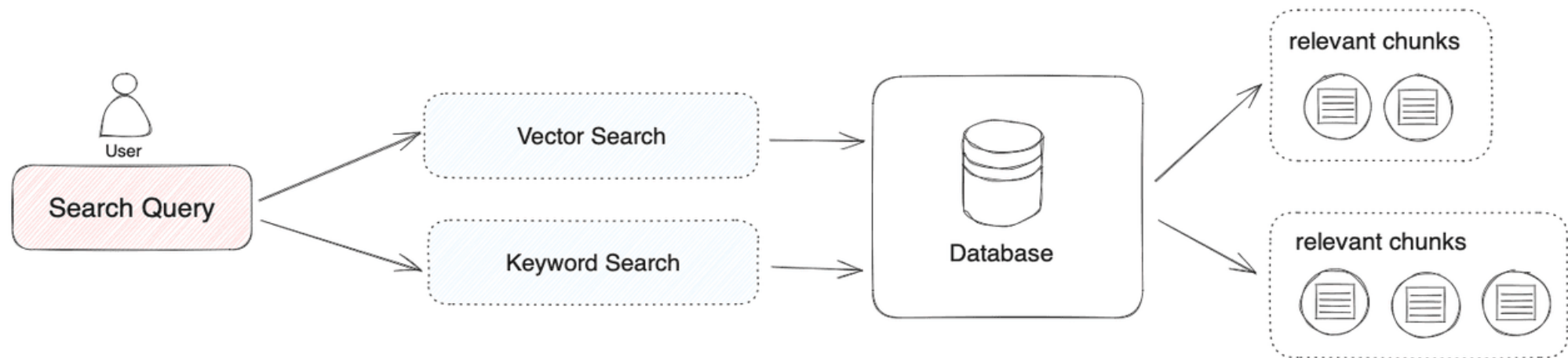
Enhancements in RAG: Query rewriting

- Refines poorly written queries for clarity and relevance.
- Removes irrelevant context for precise matching.
- Adds keywords to improve retrieval accuracy.
- Splits complex queries into manageable sub-questions.
- Generates "step-back" questions for multi-level reasoning.
- Uses advanced techniques (e.g., HyDE) to create hypothetical documents for better intent capture.

Enhancements in RAG: Hybrid search

- Use more approaches for retrieving relevant data from external source.
 - Text-mining based approaches
 - Keyword based search

Hybrid Search



Enhancements in RAG : Data preprocessing

- **Examining and Extracting Data**

- Understand dataset formats (text, PowerPoint, PDF, Excel, etc.).
- Choose the right tools for extraction based on file types.

- **Cleaning the Data**

- Remove unnecessary elements (extra whitespace, blank lines, page headers, etc.).
- Optionally convert some information into metadata to preserve document structure.

- **Chunking the Data**

- Split data into smaller, manageable pieces based on model requirements.
- Choose chunking strategies based on indexing needs and model limitations.

- **Adding Metadata**

- Add context to data chunks (e.g., page number, header, position in document).
- Ensure that metadata aids in future searches and maintains document structure.
- NER for data extraction

Enhancements in RAG: Data Chunking

- **Content Type:** Adjust strategy based on document length (e.g., articles vs. tweets).
- **Embedding Model:** Match chunk size to model capabilities (e.g., sentence-transformers for short sentences, Ada-002 for 256-512 tokens).
- **Query Characteristics:** Align chunking with user query complexity (short vs. detailed queries).
- **Application Goals:** Tailor chunk size based on use case (e.g., semantic search, summarization) and token limits for downstream tasks.

Enhancements in RAG: Data Chunking

Fixed-size Chunking

- Divides text into equal-sized chunks with optional overlap.
- Simple, efficient, and no need for NLP tools.
- General applications with minimal complexity.

Content-aware Chunking

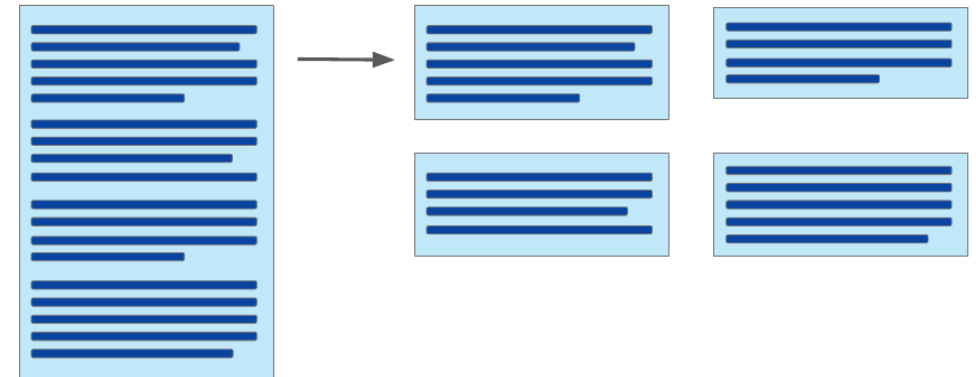
- **Sentence Splitting:** Uses tools (e.g., NLTK, spaCy) to split by sentences.
- Sentence-level tasks (e.g., classification, embeddings).

Recursive Chunking

- Iteratively divides text based on hierarchical separators (e.g., paragraphs)
- Preserves structure in complex documents.
- Nested or variable structures like reports or books.

Semantic Chunking

- Groups sentences with similar themes using semantic embeddings.
- Creates coherent, topic-based chunks.
- Semantic search, summarization, and thematic analysis.



Enhancements in RAG: Data update

- **Asynchronous Updates:** Regularly update documents and embeddings to ensure retrieval accuracy.
- **Automated Real-time Processes:** Instantly reflect new data with real-time updates for dynamic systems.
- **Periodic Batch Processing:** Perform scheduled updates (daily/weekly) for efficiency with large data volumes or less frequent changes.

